

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

COMMAND LINE CODE:

```
from math import gcd

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

p = 17
q = 11

n = p * q
phi = (p - 1) * (q - 1)

e = 7
while gcd(e, phi) != 1:
    e += 2

d = mod_inverse(e, phi)

print("Public Key :", (e, n))
print("Private Key:", (d, n))

# Encryption
def encrypt(message):
    return [pow(ord(ch), e, n) for ch in message]

# Decryption
def decrypt(cipher):
    return ''.join(chr(pow(c, d, n)) for c in cipher)

while True:
    print("\n===== RSA MENU =====")
    print("1. Encrypt")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
print("2. Decrypt")
print("3. Exit")

choice = input("Enter choice: ")

if choice == "1":
    msg = input("Enter Plain Text: ")
    cipher = encrypt(msg)
    print("Encrypted Message:")
    print(cipher)

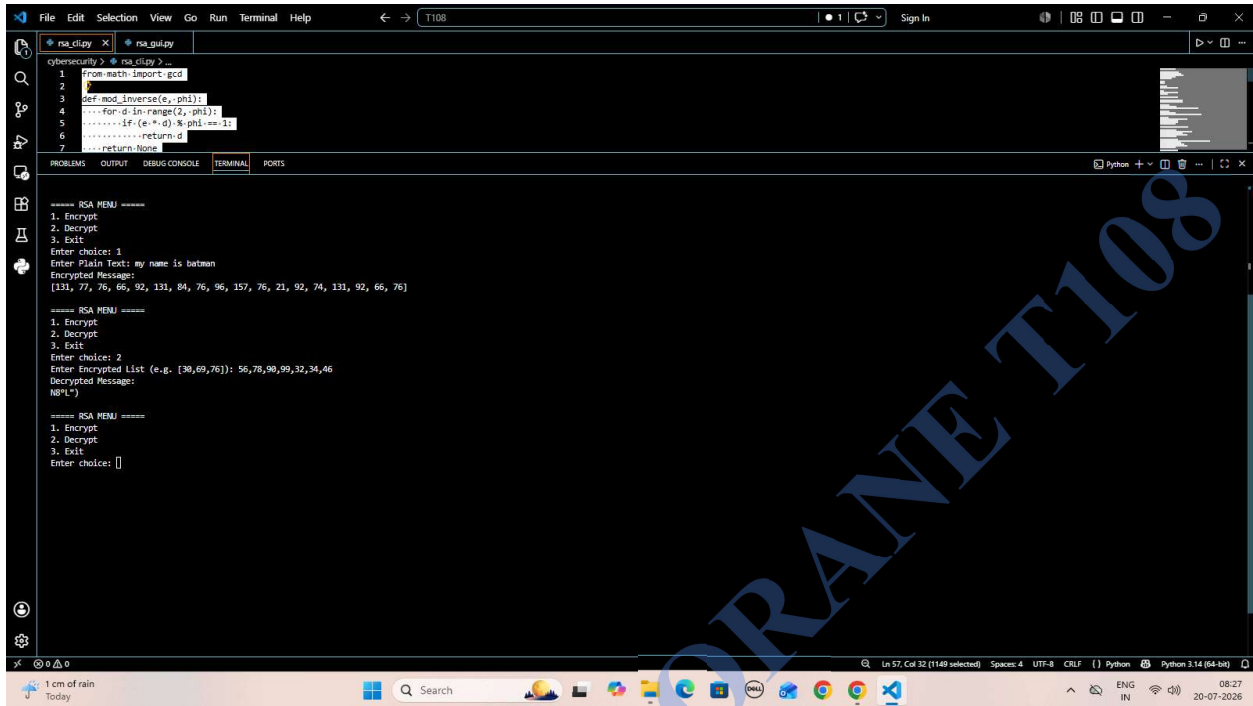
elif choice == "2":
    data = input("Enter Encrypted List (e.g. [30,69,76]): ")
    cipher = eval(data)
    plain = decrypt(cipher)
    print("Decrypted Message:")
    print(plain)

elif choice == "3":
    break

else:
    print("Invalid Choice")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

OUTPUT



The screenshot shows a Python IDE with a file named 'rsa_gui.py'. The code defines a 'mod_inverse' function and sets up RSA parameters (p=17, q=11, n=187, phi=176, e=7). It then prompts the user to choose between encryption and decryption. The output shows the encryption of the message 'my name is batman' into the list [131, 77, 76, 66, 92, 131, 84, 76, 96, 157, 76, 21, 92, 74, 131, 92, 66, 76]. The decryption of the list [38, 69, 76] results in the message '56,78,90,99,32,34,46'.

```
1 from math import gcd
2
3 def mod_inverse(e, phi):
4     for d in range(2, phi):
5         if (e * d) % phi == 1:
6             return d
7     return None
8
9 p = 17
10 q = 11
11 n = p * q
12 phi = (p - 1) * (q - 1)
13 e = 7
14 while gcd(e, phi) != 1:
15     e += 2
16
17 menu = """
18 1. Encrypt
19 2. Decrypt
20 3. Exit
21 Enter choice: """
22 choice = input(menu)
23
24 if choice == '1':
25     plain_text = input("Enter Plain Text: ")
26     encrypted_message = []
27     for char in plain_text:
28         char_int = ord(char)
29         encrypted_int = (char_int * e) % phi
30         encrypted_message.append(encrypted_int)
31     print("Encrypted Message:", encrypted_message)
32
33 elif choice == '2':
34     encrypted_list = input("Enter Encrypted List (e.g. [38,69,76]): ")
35     encrypted_list = list(map(int, encrypted_list.split(',')))
36     decrypted_message = ""
37     for int_val in encrypted_list:
38         decrypted_int = mod_inverse(e, phi)
39         decrypted_char_int = (int_val * decrypted_int) % phi
40         decrypted_char = chr(decrypted_char_int)
41         decrypted_message += decrypted_char
42     print("Decrypted Message:", decrypted_message)
43
44 elif choice == '3':
45     exit()
46
47 if __name__ == '__main__':
48     main()
49 """
```

GUI CODE:

```
import tkinter as tk
from tkinter import messagebox
from math import gcd

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

p = 17
q = 11
n = p * q
phi = (p - 1) * (q - 1)
e = 7
while gcd(e, phi) != 1:
    e += 2
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
d = mod_inverse(e, phi)

def encrypt():

    message = inputBox.get("1.0", tk.END).strip()

    if message == "":
        messagebox.showwarning("Warning", "Please enter a message.")
        return

    if message.startswith("["):
        messagebox.showinfo("Info", "Please enter plain text to encrypt.")
        return

    cipher = [pow(ord(ch), e, n) for ch in message]

    outputBox.config(state="normal")
    outputBox.delete("1.0", tk.END)
    outputBox.insert(tk.END, str(cipher))
    outputBox.config(state="disabled")

def decrypt():

    text = inputBox.get("1.0", tk.END).strip()

    if text == "":
        messagebox.showwarning("Warning", "Please enter encrypted
numbers.")
        return

    try:
        cipher = eval(text)

        message = ''.join(chr(pow(c, d, n)) for c in cipher)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
outputBox.config(state="normal")
outputBox.delete("1.0", tk.END)
outputBox.insert(tk.END, message)
outputBox.config(state="disabled")

except:
    messagebox.showerror(
        "Error",
        "Enter encrypted numbers like:\n\n[30, 69, 76, 76, 10]"
    )

def clear():

    inputBox.delete("1.0", tk.END)

    outputBox.config(state="normal")
    outputBox.delete("1.0", tk.END)
    outputBox.config(state="disabled")

root = tk.Tk()
root.title("RSA Encryption and Decryption")
root.state("zoomed")
root.configure(bg="#EAF2F8")

header = tk.Label(
    root,
    text="RSA Encryption and Decryption",
    font=("Segoe UI", 24, "bold"),
    bg="#154360",
    fg="white",
    pady=18
)
header.pack(fill="x")

frame = tk.Frame(root, bg="white", bd=2, relief="solid")
frame.pack(fill="both", expand=True, padx=35, pady=30)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
tk.Label(  
    frame,  
    text=f"Public Key : ({e}, {n})",  
    bg="white",  
    fg="#154360",  
    font=("Segoe UI", 13, "bold")  
) .pack(pady=(20,5))  
  
tk.Label(  
    frame,  
    text=f"Private Key : ({d}, {n})",  
    bg="white",  
    fg="#154360",  
    font=("Segoe UI", 13, "bold")  
) .pack()  
  
tk.Label(  
    frame,  
    text="Enter Plain Text or Encrypted Numbers",  
    bg="white",  
    font=("Segoe UI", 14, "bold")  
) .pack(pady=(30,10))  
  
inputBox = tk.Text(  
    frame,  
    height=6,  
    width=80,  
    font=("Consolas", 12),  
    bd=2,  
    relief="solid"  
)  
inputBox.pack()  
  
buttonFrame = tk.Frame(frame, bg="white")  
buttonFrame.pack(pady=25)  
  
tk.Button(  
    buttonFrame,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
text="Encrypt",
command=encrypt,
bg="#28A745",
fg="white",
font=("Segoe UI", 12, "bold"),
width=15,
height=2
).grid(row=0, column=0, padx=10)

tk.Button(
    buttonFrame,
    text="Decrypt",
    command=decrypt,
    bg="#007BFF",
    fg="white",
    font=("Segoe UI", 12, "bold"),
    width=15,
    height=2
).grid(row=0, column=1, padx=10)

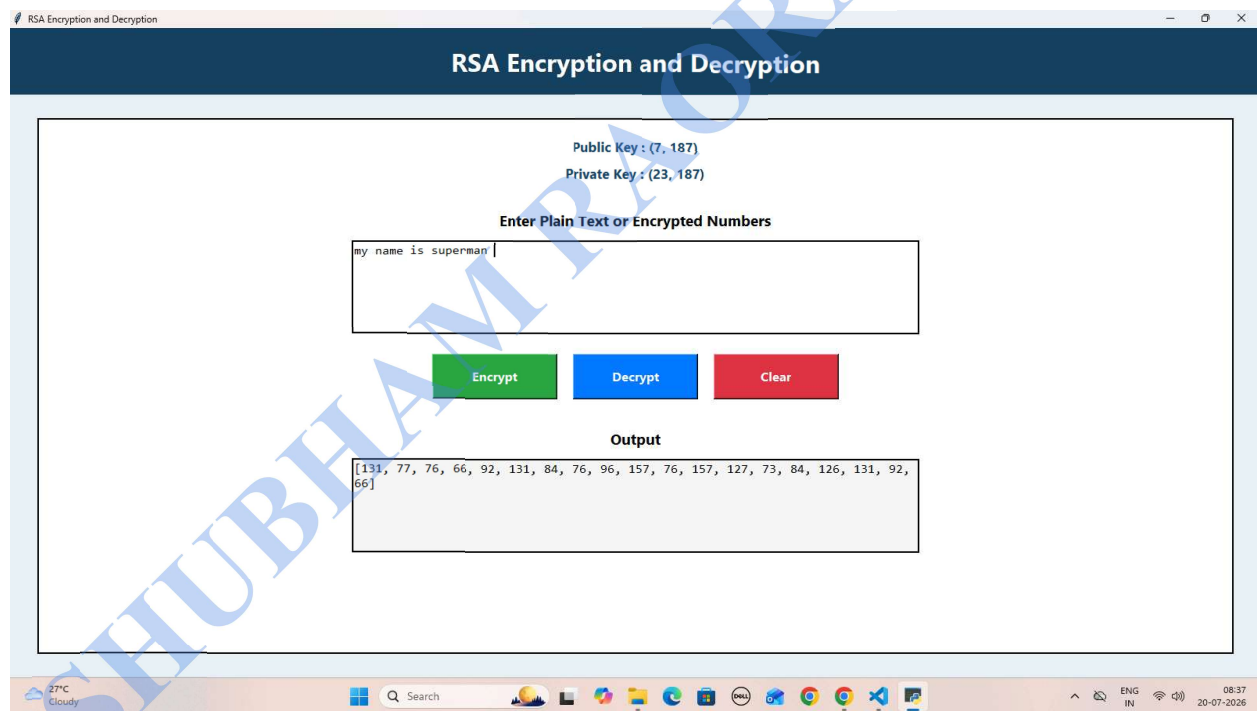
tk.Button(
    buttonFrame,
    text="Clear",
    command=clear,
    bg="#DC3545",
    fg="white",
    font=("Segoe UI", 12, "bold"),
    width=15,
    height=2
).grid(row=0, column=2, padx=10)

tk.Label(
    frame,
    text="Output",
    bg="white",
    font=("Segoe UI", 14, "bold")
).pack(pady=(10,10))
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

```
outputBox = tk.Text(  
    frame,  
    height=6,  
    width=80,  
    font=("Consolas", 12),  
    bd=2,  
    relief="solid",  
    bg="#F8F9F9",  
    state="disabled"  
)  
outputBox.pack()  
  
root.mainloop()
```

OUTPUT



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : Cyber & Information Security

